

Tutorial: Automate your SP-API Calls Using a JavaScript SDK for Node.js ↕

English

🏠 Home

☰ Documentation

<> API Reference

🔗 Code Samples

📢 Announcements

Models

Release Notes

FAQ

GitHub

Videos

GET STARTED

Welcome to SP-API Documentation

What is the Selling Partner API?

Selling Partner API Onboarding Overview >

Terminology

Frequently Asked Questions >

CHANGELOG

SP-API Release Notes

SP-API Deprecation Schedule

SP-API Product Metadata Updates

DIRECTORIES

SP-API Models

Tutorial: Automate your SP-API Calls Using a JavaScript SDK for Node.js

Automate your SP-API calls with Login with Amazon (LWA) token exchange and authentication.

This tutorial provides you with all the required details to generate a JavaScript SDK for Node.js with Login with Amazon (LWA) token exchange and authentication to build your application seamlessly. You learn the prerequisites required to build the JavaScript SDK for Node.js and also view an example using the Selling Partner API for Sellers and Swagger Code Generator.

The SP-API JavaScript SDK for Node.js is designed for server-side integration with Amazon's Selling Partner API. This tutorial covers integrating and making SP-API calls with a focus on practical application, so you can make API calls and integrate Amazon's marketplace capabilities directly into your server-side processes.

Prerequisites

To complete this tutorial, you need:

- A hybrid or SP-API application in draft or published state
- Integrated development environment (IDE) software
- Java version 7 or higher
- `swagger-codegen-cli-2.4.29`
- Node.js v18 or higher

! Warning

You must use `swagger-codegen-cli-2.4.29`. If you use `swagger-codegen-cli` versions other than **2.4.29**, generating the SDK will fail. Using the wrong version could cause larger issues in the integration process.

Before your application can connect to the Selling Partner API, you must register it, and it must be authorized by a selling partner. If you do not have a hybrid or SP-API app, follow the steps to [register as a developer](#), [register your application](#), and [Authorizing Selling Partner API applications](#). Then, return to this tutorial.

Next, set up your workspace for the tutorial.

Step 1. Set up your workspace

1. On your local drive, create a directory for this project, name it **SPAPI_JavaScript_SDK**, and navigate to the new directory.

```
Bash
```

SP-API Seller Use Cases

SP-API Vendor Use Cases

Seller Central URLs

Vendor Central URLs

REGISTRATION

SP-API Registration Overview >

Developer Registration Request Status

Third-Party Provider Registration >

Service Provider Registration and Role Approval Process Overview

Website Guidelines for Public Developers and Service Providers

User Permissions for Service Providers

Register your Application

Update your Application Information

Rotate your Application's LWA Credentials >

View your Application Information and Credentials

Learn How Sellers Authorize Service Providers

Selling Partner API Roles >

Policies and Agreements

About Facial Data

AUTHORIZATION

Authorize Applications >

Renew Authorizations

Revoke Authorizations

Authorization Limits

Special Authorization Types >

INTEGRATION

Building Robust Amazon SP-API Applications

SP-API SDKs >

```
mkdir SPAPI_JavaScript_SDK
cd SPAPI_JavaScript_SDK
```

2. Download the following tools.

1. IDE software (this walkthrough uses [Visual Studio](#) IDE on Windows OS)
2. [GNU Wget](#)
3. [Java version 7 or higher](#)

3. Run the following command to download the Swagger Code Generator:

Bash

```
wget https://repo1.maven.org/maven2/io/swagger/swagger-codegen-cli/2.4.29/swagger-codegen-cli-2.4.29.jar
```

4. Use the following command to get the file and change permissions to make it executable.

Bash

```
chmod 744 swagger-codegen-cli-2.4.29.jar
```

5. Copy `swagger-codegen-cli.jar` into your local directory

```
C:\SPAPI_JavaScript_SDK .
```

6. In GitHub, go to <https://github.com/amzn/selling-partner-api-models/tree/main/models> and use the following command to clone the `selling-partner-api-models` repository to your local directory `C:\SPAPI_JavaScript_SDK`.

Bash

```
git clone https://github.com/amzn/selling-partner-api-models
```

Now that you have completed the required setup, the next step is to generate the JavaScript SDK for Node.js with the authentication and authorization classes provided in the [sellingpartner-api-javascript](#) folder.

Step 2. Generate a JavaScript SDK for Node.js with LWA token exchange and authentication

Navigate to the `SellingPartnerAPISdkJavaScriptNode/src` directory. In the `src` directory find the `generate-js-sdk.sh` shell script file. Run the following script in the command line:

Bash

```
cd SellingPartnerAPISdkJavaScriptNode/src
./generate-js-sdk.sh -j ../../swagger-codegen-cli-2.4.29.jar
```

You can now view the `models` and `sdk` directories, which contain the API models and the JavaScript SDK respectively, in your package root.

This folder provides helper classes to generate an [access token](#) for Amazon Selling Partner APIs. It is intended for use with the Selling Partner API Client Libraries generated by [Swagger Codegen](#) using the RestSharp library. It can also be integrated into custom projects.

Step 3. Create a local npm package

Package your SDK to use across multiple local projects without publishing the SDK to the public npm registry.

- 1. Open your `package.json` file and add the following to ensure that the `package.json` is properly set up to include the necessary directories.:

```
JSON

{
  "files": [
    "src/helper/",
    "sdk/"
  ]
}
```

This step ensures that only the specified directories and their contents (for example, `src/helper/` and `sdk/`) are included in the package, and others are ignored.

- 2. Navigate to the `SellingPartnerAPISdkJavaScriptNode` directory in your terminal or command prompt.
- 3. Run the following command to create the package:

```
Bash

npm pack
```

This command creates a tarball (TGZ file) in your project directory, named `amzn-testsellingpartnerjavascriptapilwalib-1.0.0.tgz` or similar, based on your package's name and version. It includes all the necessary files from your project, effectively creating a local package that can be installed just like any published npm package.

Step 4. Create a new project using the local npm package

Now you can create a new project that uses the local npm package. With this approach, you can test the SDK in an environment similar to what end developers will encounter.

- 1. Enter the following command to create a new directory for the new project. This example assumes your current directory is `SellingPartnerAPISdkJavaScriptNode/src` :

```
Bash

cd ../../
mkdir MyNewSPAPIProject
cd MyNewSPAPIProject
```

- 2. Set up a new Node.js project to ensure a clean environment.

```
Bash

npm init -y
```

- 3. Follow the prompts to initialize your new project. Accept the default options if they match your preferences.
- 4. Install the local npm package you created in [Step 3. Create a local npm package](#) into this new project. For the following command, replace the placeholder `../SellingPartnerAPISdkJavaScriptNode/amzn-testsellingpartnerjavascriptapilwalib-1.0.0.tgz` with the actual path to your TGZ file. This step is crucial because it simulates how other developers will integrate your SDK into their projects.

```
Bash

npm install ../SellingPartnerAPISdkJavaScriptNode/amzn-testsellingpartnerjavascriptapilwalib-1.0.0.tgz
```

Delete an Application From Your Developer Account

CALLING THE SP-API

Configuration Details

Call Structure

Development Tools

Performance Management

SOLUTION PROVIDER PORTAL

Manage Users in Solution Provider Portal

Manage Your Business And Contact information in Solution Provider Portal

Update Your Developer Profile in Solution Provider Portal

Unify Your Developer Accounts

Verify Your Identity in Solution Provider Portal

API Usage Metrics in Solution Provider Portal

TUTORIALS

Tutorial: Subscribe to the ORDER_CHANGE Notification

[Tutorial: Test Selling Partner API Endpoints](#)[Tutorial: Automate your SP-API Calls Using a C# SDK](#)[Tutorial: Automate your SP-API Calls Using a Java SDK](#)[Tutorial: Automate your SP-API Calls Using a JavaScript SDK for Node.js](#)[Tutorial: Automate your SP-API Calls Using a Python SDK](#)[Tutorial: Automate your SP-API Calls using a prebuilt C# SDK](#)[Tutorial: Automate your SP-API Calls using prebuilt Java SDK](#)[Tutorial: Automate your SP-API Calls using a prebuilt JavaScript SDK](#)[Tutorial: Automate your SP-API Calls using prebuilt PHP SDK](#)[Tutorial: Automate your SP-API calls using a prebuilt Python SDK](#)[Tutorial: Authorize Multiple Vendor Central Accounts with a Single SP-API Application](#)[Tutorial: Retrieve Merchant Shipping Templates](#)[Tutorial: Grant the SP-API Permission to an Amazon SQS Queue](#)[Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier](#)

TROUBLESHOOTING

[Troubleshoot SP-API Errors](#)[URL Encoding](#)[Resolve Common HTTP and Authorization Error Codes](#)[External Fulfillment Errors](#)[Listings Items API Issues Troubleshooting](#)

SELLING PARTNER APPSTORE

[What is the Selling Partner Appstore?](#)[List Your App on the Selling Partner Appstore](#)[Edit Your Appstore Listing](#)[Check Listing Status](#)[Appstore Ratings and Reviews](#)[Press Releases and Promotions](#)

SERVICE PROVIDER NETWORK

[List Your Service](#)

SECURITY AND COMPLIANCE

[SP-API Security and Compliance Overview](#)[Amazon Selling Partner API Guard Implementation Guide](#)

Note

The path to your `.tgz` file can be absolute or relative, based on your current working directory. For example, if your `.tgz` file is in the `SellingPartnerAPISdkJavaScriptNode` directory, your command is

```
npm install ../SellingPartnerAPISdkJavaScriptNode/amzn-testsellingpartnerjavascriptapilwalib-1.0.0.tgz
```

After completing these steps, your package will be listed as a dependency in your new project's `package.json`, and you can import functionalities as you normally would.

Step 5. Connect to the Selling Partner API using the generated JavaScript SDK for Node.js

With authentication set up, you're now ready to interact with the SP-API endpoints.

1. Store your LWA credentials in an `app.config.mjs` file within the project root.

Bash

```
touch app.config.mjs
```

2. Define your credentials inside `app.config.mjs` as follows:

JavaScript

```
export const AppConfig = {
  lwaClientId: "< LWA client ID >",
  lwaClientSecret: "< LWA client secret >",
  lwaRefreshToken: "< LWA refresh token >",
}
```



Caution

Never commit this file to your version control system as it contains sensitive information. Please ensure these LWA credentials are stored securely in an encrypted format.

3. Create a new file named `index.js` in your project root directory and insert the following script, which initiates a call to a specific SP-API endpoint and logs the response to the console.

Bash

```
touch index.js
```

4. Copy and paste the following into `index.js`:

JavaScript

```
import { AppConfig } from './app.config.mjs';
import { LwaAuthClient } from '@amzn/testsellingpartnerjavascriptapilwalib/src/helper/LwaAuthClient.mjs';

import {
  SellersApi,
  ApiClient as SellersApiClient,
} from '@amzn/testsellingpartnerjavascriptapilwalib/sdk/src/sellers/index.js';

(async () => {
```

VAT Calculation Service



```
const lwaClient = new LwaAuthClient(
  AppConfig.lwaClientId,
  AppConfig.lwaClientSecret,
  AppConfig.lwaRefreshToken
);
const sellerApiClient = new SellersApiClient(
  'https://sellingpartnerapi-fe.amazon.com'
);

const sellerApi = new SellersApi(sellerApiClient);
sellerApiClient.applyXAmzAccessTokenToRequest(
  await lwaClient.getAccessToken()
);
const participations = await
sellerApi.getMarketplaceParticipations();
console.log(
  JSON.stringify(participations, null, ' ') +
  '\n*****'
);
})();
```

5. Run the script: Run the following command in your terminal or command prompt to initiate a GET request to the Sellers API endpoint:

Amazon Seller Data Access
Best Practices



```
Bash
node index.js
```

This command runs your script, initiates the API call, and displays the results in your console.

Expected Output:

A+ CONTENT API
A+ Content API



A+ Content Examples

AMAZON WAREHOUSING AND DISTRIBUTION API
Amazon Warehousing and Distribution API



```
JSON
{
  "payload": [
    {
      "marketplace": {
        "id": "xxxxx",
        "countryCode": "JP",
        "name": "Amazon.co.jp",
        "defaultCurrencyCode": "JPY",
        "defaultLanguageCode": "ja_JP",
        "domainName": "www.amazon.jp"
      },
      "participation": {
        "isParticipating": true,
        "hasSuspendedListings": false
      }
    }
  ]
}
```

Updated 18 days ago

Amazon Warehousing and Distribution API v2024-05-09 Reference

← Tutorial: Automate your SP-API Calls Using a Java SDK

Tutorial: Automate your SP-API Calls Using a Python SDK →

APP INTEGRATIONS API
App Integrations API



Did this page help you?

APPLICATION MANAGEMENT API

Application Management API >

CATALOG ITEMS API

Catalog Items API >

Catalog Items API Rate Limits

Catalog Items API v2022-04-01 Reference

CUSTOMER FEEDBACK API

Customer Feedback API >

Customer Feedback API Rate Limits

DATA KIOSK

Data Kiosk API >

Data Kiosk Schema Explorer User Guide

Data Kiosk Query Processing Finished Notification

Building Data Kiosk workflows guide

Vendor Analytics Dataset Use Case Guide

Schedule Data Kiosk Queries

DELIVERY BY AMAZON API

Delivery by Amazon API >

EASY SHIP API

Easy Ship API >

EXTERNAL FULFILLMENT

External Fulfillment Inventory API >

External Fulfillment Returns API >

External Fulfillment Shipping API >

External Fulfillment Errors
External Fulfillment APIs Rate Limits

FULFILLMENT BY AMAZON (FBA)

FBA Inventory API >

FBA Inventory Dynamic Sandbox Guide

FEEDS API

Feeds API >

Feed Type Values >

Feeds JSON Schemas ↗

Feeds API Best Practices

Feeds API FAQ

Feeds API Rate Limits

FINANCES

Finances API >

Transfers API >

FULFILLMENT INBOUND API

Fulfillment Inbound API >

Fulfillment Inbound API FAQ

Migrating Fulfillment Inbound workflows

Fulfillment Inbound API Rate Limits

FULFILLMENT OUTBOUND API

Fulfillment Outbound Dynamic Sandbox Guide

Fulfillment Outbound API >

Fulfillment Outbound API Rate Limits

MCF Best Practices

INVOICING

Invoices API >

Invoices API FAQ

LISTINGS

Listings Items API >

Listings Items API Rate Limits

Listings Restrictions API >

Listings Restrictions API Rate Limits

Manage Product Listings with the Selling Partner API

Building Listings Management Workflows Guide

Understanding Amazon listing status and seller-fulfilled inventory management

Listings Management Workflow Migration >

Manage Amazon Haul, Advanced Multiple-Offer, and Multiple-Fulfillment Use Cases

Listings APIs FAQ

Product Type Definitions API >

Product Type Definitions API Rate Limits

MERCHANT FULFILLMENT API

Merchant Fulfillment API >

MESSAGING API

Messaging API >

NOTIFICATIONS API

Notifications API >

Notification Type Values

Tutorial: Subscribe to the ORDER_CHANGE Notification ↗

ORDERS API

Orders API >

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier ↗

Orders API Migration Guide

Orders API Rate Limits

PRODUCT FEES API

Product Fees API >

PRODUCT PRICING API

Product Pricing API >

Product Pricing API and Notifications FAQ

Price Adjustment Automation Workflows Guide
Manage automated pricing rules with SP-API

REPLENISHMENT API

Replenishment API >

REPORTS API

Reports API >

Report Type Values >

Reports JSON Schemas ↗

Reports API FAQ

SALES API

Sales API >

SELLER WALLET API

Seller Wallet API >

Seller Wallet API Rate Limits

SELLERS API

Sellers API >

SERVICES API

Services API >

SHIPMENT INVOICING API

Shipment Invoicing API >

SHIPPING API

Shipping API v2 Resources ↗

Shipping API v1 Reference

SOLICITATIONS API

Solicitations API >

SUPPLY SOURCES API

Supply Sources API >

Multi-Location Inventory Integration Guide

Supply Sources API Rate Limits

TOKENS API

Tokens API >

UPLOADS API

Uploads API >

VEHICLES API

Vehicles API >

Vehicles API Rate Limits

VENDOR DIRECT FULFILLMENT APIS

Vendor Direct Fulfillment Dynamic Sandbox Guide

Vendor Direct Fulfillment Workflow Guide

SP-API Bill-to-Party Addresses

VENDOR DIRECT FULFILLMENT TRANSACTION STATUS API

Vendor Direct Fulfillment Transaction Status API >

VENDOR DIRECT FULFILLMENT INVENTORY API

Vendor Direct Fulfillment Inventory API >

VENDOR DIRECT FULFILLMENT ORDERS API

Vendor Direct Fulfillment Orders API >

VENDOR DIRECT FULFILLMENT SHIPPING API

Vendor Direct Fulfillment Shipping API >

VENDOR DIRECT FULFILLMENT PAYMENTS API

Vendor Direct Fulfillment Payments API >

VENDOR RETAIL PROCUREMENT INVOICES API

Vendor Retail Procurement Invoices API >

VENDOR RETAIL PROCUREMENT ORDERS API

Vendor Retail Procurement Orders API >

VENDOR RETAIL PROCUREMENT SHIPMENTS API

Vendor Retail Procurement Shipments API >

**VENDOR RETAIL PROCUREMENT TRANSACTION STATUS
API**

Vendor Retail Procurement Transaction Status API >

Powered
by

